UNIVERSIDAD DE EL SALVADOR

Ingeniería en Desarrollo de Software · Educación en Línea

Enfoque, decisiones, reutilización e interacción del sistema

Tema 6. Sistema de microservicios con Kafka para el procesamiento de datos en tiempo real. Prototipo: tienda virtual de relojes.

Proyecto base reutilizado: TiendaRelojes (proyecto_x).

Fecha del documento: 24 de octubre de 2026.

Integrantes: completar antes de la entrega.

Este documento explica cómo está construida la tienda, por qué se tomaron esas decisiones, qué partes se reutilizaron del proyecto base y cómo colaboran el navegador, los tres servicios y Kafka cuando una persona compra un reloj. El texto describe el sistema tal como queda en el repositorio, no una arquitectura distinta a la que se puede ejecutar.

1. Contexto y objetivo

Una tienda en línea necesita mostrar un catálogo, aceptar un pedido, comprobar que hay piezas y registrar el resultado del cobro. Si todo eso vive en un solo programa y en una sola base de datos, cualquier cambio de inventario arrastra al resto. El Tema 6 pide separar esas responsabilidades en microservicios y comunicar los cambios con Apache Kafka, de modo que el procesamiento del pedido ocurra a partir de eventos.

El objetivo de este prototipo es recorrer una venta de principio a fin: elegir un reloj, crear el pedido, reservar existencias, simular el pago y ver cómo el stock y el estado del pedido cambian sin que los servicios compartan tablas. El cobro es una simulación. No hay banco, no hay cargo real y la aplicación no está preparada para publicarse en Internet.

El resultado que se puede demostrar es concreto:

- Cuatro relojes iniciales, con precio y disponibilidad.
- Una cesta visible junto al catálogo y un formulario de pedido en la misma página.
- Tres APIs ASP.NET Core 8: inventario, pedidos y pagos.
- Un tópico Kafka, shop-events, por el que circulan los hechos de la compra.
- Una base SQLite propia para cada servicio, conservada en un volumen de Docker.
- Un solo acceso para el navegador: Nginx en el puerto 8088.

2. Enfoque utilizado

2.1. La compra como una secuencia de hechos

El enfoque es de coreografía. Ningún servicio dirige a los demás ni les hace una llamada para ordenarles el siguiente paso. Quien termina una parte de la compra publica un hecho. Quien necesita enterarse lo lee y decide, con su propia regla, qué hacer. El pedido no queda cerrado en la misma petición HTTP que lo crea: esa petición solo lo registra y anuncia que existe.

Esa forma se parece al patrón Observador, con una diferencia práctica. El publicador no guarda la lista de interesados. Kafka conserva el mensaje y cada grupo de consumidores recibe su copia. Agregar otro interesado, por ejemplo un servicio de notificaciones, no obliga a modificar a quien publicó el pedido.

2.2. Dos canales, cada uno con un motivo

Kafka transporta los cambios de negocio: pedido creado, stock reservado o rechazado, pago confirmado o fallido, pedido cancelado. HTTP se reserva para lo que necesita una respuesta en el momento. Pedidos consulta el catálogo para copiar el precio que tiene el servidor. Pagos consulta el estado del pedido antes de aceptar la simulación. El navegador pide productos, pedidos y pagos, y nunca abre una conexión con el broker.

La página se entera del avance preguntando el pedido cada 2,5 segundos mientras la ventana de seguimiento está abierta. Esa consulta es a propósito. Meter Kafka en el navegador obligaría a exponer el broker y a convertir la interfaz en otro servicio. Con el sondeo, la pantalla sigue el mismo estado que ya guardó OrdersApi.

2.3. Cada servicio es dueño de sus datos

Inventario no lee la tabla de pedidos. Pedidos no modifica el stock. Pagos no descuenta piezas. El único dato que cruzan es el mensaje, y el identificador del pedido es el hilo que los une. Así se cumple la idea de microservicio que pide el tema: se puede explicar cada programa sin abrir el código de los otros, y se puede reiniciar uno sin borrar los datos de los demás.

El navegador tampoco es una cuarta base. La cesta vive en la memoria de la página hasta que la persona pulsa «Crear pedido». Antes de ese clic no hay evento, no hay reserva y otro cliente todavía puede comprar las mismas unidades.

Pieza	Responsabilidad	Qué no hace
Tienda (HTML, CSS y JavaScript)	Catálogo, cesta, formulario, seguimiento y administración.	No publica en Kafka ni calcula el precio definitivo.
Nginx	Sirve la página y reenvía /api/inventory, /api/orders y /api/payments.	No contiene reglas de negocio.
InventoryApi	Productos, stock, reservas y movimientos.	No guarda clientes ni el total del pedido.
OrdersApi	Pedidos, estados y el aviso OrderPlaced.	No aparta ni descuenta existencias.
PaymentsApi	Registro del cobro simulado.	No habla con un banco ni modifica el stock.
Kafka	Conserva y reparte los hechos del tópico shop-events.	No decide si hay stock ni si el pago procede.
Shared	Contratos de eventos y el cliente Kafka usado por los tres servicios.	No tiene base de datos ni endpoints.

Tabla 1. Responsabilidad de cada parte del sistema.

3. Decisiones tomadas

Las decisiones siguientes están en el código que se ejecuta. Cada una se tomó para que el prototipo se pueda explicar y levantar con Docker en una sola máquina.

3.1. Conservar los tres servicios del proyecto base

El proyecto TiendaRelojes ya separaba inventario, pedidos y pagos en tres APIs ASP.NET Core, con Entity Framework Core y SQLite. El tema pide microservicios, así que esa división se mantuvo. Rehacerlos como un monolito o fundir las bases habría ido en contra del enunciado. Los archivos Program.cs de los tres servicios, sus Dockerfiles, compose.yaml y nginx.conf siguen siendo los del proyecto base.

3.2. Un solo tópico y un campo Type

Todos los hechos viajan por `shop-events`. El sobre es el registro `EventMessage`: un identificador, el tipo, los datos y la fecha. El campo `Data` va como texto JSON dentro del mensaje. El consumidor lee primero el sobre y, según `Type`, interpreta el cuerpo como `OrderPlaced`, `StockReserved` u otro contrato.

Un solo tópico hace visible el flujo en un broker de demostración. Cada consumidor recibe todos los mensajes y actúa solo cuando el tipo le corresponde. El resto lo registra como ya visto, para no aplicarlo dos veces, y confirma el offset.

3.3. Tres grupos de consumidores

Kafka entrega una copia del mensaje a cada grupo. Por eso los grupos se llaman `inventory-service`, `orders-service` y `payments-service`. Si los tres compartieran grupo, solo uno procesaría cada pedido y la venta se quedaría a medias. Dentro de cada grupo hay un solo proceso, así que no hace falta repartir particiones entre varias instancias.

3.4. Confirmación manual y lectura desde el principio

El consumidor se crea con confirmación automática desactivada y con `AutoOffsetReset.Earliest`. El offset se confirma después de guardar en SQLite. Si el proceso se detiene antes de confirmar, el mensaje vuelve a leerse. Empezar desde el principio permite que un servicio que arranca tarde todavía vea hechos que ya se publicaron durante el arranque.

3.5. Crear el tópico al arrancar

En el proyecto base, publicar y suscribirse daba por hecho que `shop-events` ya existía. En un broker nuevo el tópico no está hasta que alguien lo crea, y el consumidor fallaba al suscribirse. `Events.EnsureTopic` lo crea con una partición y factor de réplica 1 antes de publicar o de escuchar. Si el tópico ya existe, la operación se ignora. Una partición basta para la demostración y conserva el orden de publicación. El factor 1 corresponde al único broker de Compose.

3.6. Reservar antes de cobrar, descontar solo si el pago procede

Disponible significa stock menos unidades ya reservadas. `OrderPlaced` no resta el stock de inmediato. Si hay piezas, inventario suma `Reserved` y guarda una fila de reserva. La salida real, el movimiento negativo y el estado `sold` ocurren cuando llega `PaymentConfirmed`. Si el pago se rechaza o el pedido se cancela, la reserva pasa a `released` y esas unidades vuelven a estar disponibles. Así dos compras simultáneas no venden la misma pieza, y un pago fallido no deja el inventario bajado.

3.7. El precio lo copia pedidos desde inventario

El navegador envía el identificador del producto y la cantidad. OrdersApi llama por HTTP a `GET /products`, agrupa líneas repetidas y copia el nombre y el precio de ese momento. El total se calcula en el servidor. La cesta puede mostrar un subtotal para la persona, pero ese número no es el que se guarda.

3.8. Cobro simulado y estados que no retroceden

PaymentsApi acepta `approved` o `rejected` en `POST /payments/{id}/simulate`. Antes consulta el pedido. Solo continúa si el estado es `awaiting-payment` y el pago sigue `pending`. Un segundo intento sobre el mismo pago se rechaza. El pedido solo cambia de estado cuando el estado actual es el que esa transición espera: de verificación a reservado, de reservado a pagado o a pago fallido. Un pedido ya cancelado no se reabre con un mensaje tardío.

3.9. SQLite por servicio y productor con acuse

Cada API crea su archivo con `EnsureCreated` la primera vez y lo guarda en un volumen Docker. Reiniciar los contenedores conserva los datos. Borrarlos exige `docker compose down -v`. El productor usa `Acks.All` e idempotencia, de modo que un reintento del cliente Kafka no duplique el mismo envío hacia el broker. La clave del mensaje es el identificador del evento.

3.10. Cesta visible y una sola puerta de entrada

La tienda del proyecto base ya tenía catálogo, búsqueda, ficha, seguimiento y administración. La compra dependía de un cuadro que no dejaba claro si el reloj había entrado en la cesta. La interfaz quedó con el botón «Añadir a la cesta» en cada tarjeta y con el formulario al lado del catálogo. Nginx sigue siendo la única puerta: el navegador habla con el puerto 8088 y las rutas `/api/...` se reenvían al servicio que corresponde. Los archivos estáticos salen del mismo Nginx.

3.11. Lo que se dejó fuera

No hay caja de salida transaccional, caducidad de reservas, autenticación, autorización ni conciliación automática. El orden actual guarda el identificador del evento y después publica la respuesta. Si el proceso muere después de guardar y antes de confirmar el offset, una reentrega puede darse por procesada y el hecho de respuesta no llegar a publicarse. Es una limitación conocida del prototipo. Para una tienda real harían falta esa caja de salida, seguridad, auditoría y pruebas de concurrencia. Aquí se documenta para que la demostración no se presente como un sistema de producción.

4. Cómo se reutilizó el código

El trabajo parte de TiendaRelojes, dentro de proyecto_x. Reutilizar, en este caso, significa conservar la estructura que ya resolvía el catálogo, el pedido y el cobro, y completar lo que impedía explicar y ejecutar el flujo con Kafka.

4.1. Qué se conservó del proyecto base

Elemento conservado	Uso que se le sigue dando
InventoryApi/Program.cs	Catálogo, ajustes, reservas, movimientos y el trabajador que escucha Kafka.
OrdersApi/Program.cs	Alta y consulta de pedidos, cancelación y el trabajador que cambia estados.
PaymentsApi/Program.cs	Consulta y simulación del pago, y el trabajador que abre el pago pendiente.
Shared/Messaging.cs	Contratos, sobre del evento, publicación y consumo. Se amplió, no se reemplazó.
compose.yaml, Dockerfiles y nginx.conf	Levantan Kafka, las tres APIs y la tienda, y enrutan /api.
frontend y las ilustraciones SVG	Catálogo, búsqueda, ficha, seguimiento y panel de administración.
docs de arquitectura, manual, informe y pruebas	Siguen en el repositorio. Este PDF los amplía con el enfoque pedido.

Tabla 2. Piezas del proyecto base que siguen en la solución.

4.2. La biblioteca Shared, usada por los tres

Shared es el único código de integración. Los tres proyectos lo referencian y llaman a los mismos miembros. Ningún servicio construye su propio productor ni redefine el nombre del tópico.

- `OrderPlaced`, `StockReserved`, `StockRejected`, `PaymentConfirmed`, `PaymentFailed` y `OrderCancelled` fijan el contenido de cada hecho.
- `EventMessage` es el sobre común: identificador, tipo, datos y fecha.
- `Events.Json` y `Events.Parse` serializan ese sobre y el cuerpo.
- `Events.Publish` envía al tópico. Lo llaman pedidos al crear o cancelar, inventario al reservar o rechazar, y pagos al simular.
- `Events.Consumer` suscribe al grupo que recibe cada trabajador.
- `Events.EnsureTopic` prepara el tópico. Esta operación se añadió sobre la biblioteca del proyecto base, porque sin ella un broker vacío detenía a los consumidores.

Si cambia el nombre del tópico o la forma del sobre, el cambio se hace en un archivo. Los servicios siguen publicando el mismo tipo y el mismo objeto de dominio.

4.3. Un patrón repetido, con reglas distintas

Los tres programas siguen la misma forma: API mínima, contexto de Entity Framework, SQLite, `EnsureCreated`, CORS abierto para la demostración, `GET /health` y un `BackgroundService` que consume Kafka en un bucle con reintento. Esa forma se reutiliza. La regla de negocio no se metió en Shared, porque cada uno responde a hechos distintos:

- InventoryWorker reserva, rechaza, confirma la venta o libera.
- OrderWorker traduce el hecho al estado del pedido.
- PaymentsWorker abre el pago pendiente cuando el stock quedó reservado.

Cada base tiene su tabla de eventos ya vistos: `ProcessedInventoryEvent`, `ProcessedOrderEvent` y `ProcessedPaymentEvent`. Las tres usan un índice único sobre el identificador del evento. El registro vive en la base del servicio porque la idempotencia es local. Que inventario ya

haya aplicado un mensaje no significa que pedidos también lo haya hecho.

4.4. Qué no se compartió

`Product`, `ShopOrder` y `Payment` no están en Shared. Poner esas entidades en la biblioteca común empujaría a una sola base y acoplaría los servicios por dentro. El navegador tampoco referencia la biblioteca de Kafka: solo conoce las rutas HTTP. La cesta, el catálogo y la administración se reutilizaron en la página que ya existía; el cambio de interfaz se limitó a dejar la cesta y el alta del pedido visibles en la misma vista.

5. Cómo interactúa cada parte

5.1. Arranque

`docker compose up --build` levanta Kafka, las tres APIs y Nginx. Las APIs esperan a que el broker responda el chequeo de salud. Al iniciar, cada una crea su SQLite si no existe. Inventario inserta cuatro relojes Casio solo cuando la tabla de productos está vacía: Rose Gold Diamond, Old Money Plata, Classic Dorado y Classic Plata Azul, todos con precio 32. Los tres trabajadores esperan tres segundos, aseguran el tópico y se suscriben. Nginx publica la tienda en el puerto 8088. Las rutas quedan así: `/api/inventory/` hacia el inventario, `/api/orders/` hacia pedidos y `/api/payments/` hacia pagos. Nginx quita el prefijo al reenviar, de modo que el servicio recibe `/products` o `/orders`.

5.2. Catálogo y cesta

El navegador pide `GET /api/inventory/products`. La tarjeta muestra el precio y si queda disponible, calculado como stock menos reservado. Si esa cuenta es cero, el botón queda deshabilitado. «Añadir a la cesta» solo guarda la selección en la página. Pulsar la tarjeta abre la ficha; pulsar el botón no abre la ficha, porque el clic se detiene en el botón. En este momento inventario, pedidos, pagos y Kafka no intervienen.

5.3. Creación del pedido

- La cesta envía `POST /api/orders/orders` con nombre, teléfono, dirección, forma de pago, identificadores y cantidades.
- OrdersApi rechaza datos vacíos, más de 30 líneas o cantidades fuera del rango de 1 a 100.
- Pide el catálogo a inventario, agrupa productos repetidos y copia nombre y precio.
- Guarda el pedido en orders.db con estado `pending-stock` y un identificador GUID. Responde 202 Accepted, porque la reserva todavía no ocurrió.
- Publica `OrderPlaced` con el identificador y las líneas.

Si Kafka no acepta el mensaje, el pedido queda guardado y la API responde 503. No hay reintento automático. La página muestra el error y la persona puede volver a intentarlo.

5.4. Reserva de inventario

InventoryWorker lee `OrderPlaced`. Si ese identificador de evento ya está en su tabla de procesados, confirma el offset y no repite la reserva. Si el pedido trae líneas y cada una cabe en el disponible, suma `Reserved`, inserta una reserva por producto y publica `StockReserved`. Si falta stock, no aparta nada y publica `StockRejected` con el motivo. Después guarda el evento como procesado y confirma el offset.

Ese resultado lo leen los otros dos servicios:

- OrderWorker, si el pedido sigue en `pending-stock`, lo pasa a `awaiting-payment` o a `rejected`.
- PaymentsWorker, al ver `StockReserved`, inserta un pago en estado `pending` si ese pedido todavía no tiene uno. Un rechazo de stock no crea pago.

5.5. Seguimiento y simulación del pago

La ventana de seguimiento llama `GET /api/orders/orders/{id}` al abrirse y otra vez cada 2,5 segundos. Los botones de pago se habilitan solo en `awaiting-payment`. Al pulsar uno, el navegador envía `POST /api/payments/payments/{id}/simulate` con `approved` o `rejected`. PaymentsApi vuelve a leer el pedido por HTTP, comprueba el estado y que el pago siga pendiente, guarda el monto y publica `PaymentConfirmed` o `PaymentFailed`.

A partir de ese hecho, cada servicio toca solo su base:

- OrdersApi pasa el pedido a `paid` o a `payment-failed`, siempre que todavía esté pendiente de pago.
- InventoryApi busca las reservas en estado `reserved`. En un pago confirmado resta el stock, baja lo reservado, marca la reserva como `sold` y escribe un movimiento negativo con el motivo «Venta» y el identificador del pedido. En un pago fallido solo baja lo reservado y marca la reserva como `released`. El stock físico no cambia.

Cuando la página ve pagado, rechazado o pago fallido, vuelve a cargar el catálogo para mostrar las existencias nuevas.

5.6. Cancelación y administración

`POST /api/orders/orders/{id}/cancel` solo acepta pedidos en verificación o pendientes de pago. OrdersApi marca `cancelled` y publica `OrderCancelled`. InventoryApi libera las reservas igual que en un pago fallido. OrderWorker ignora mensajes posteriores de un pedido ya cancelado, de modo que una reserva que llegue tarde no lo devuelve a pendiente de pago.

El panel de administración no usa Kafka. Lista movimientos, ajusta existencias con `POST /products/{id}/adjust` y registra un reloj con `POST /products`. Esas operaciones son de inventario y ocurren por HTTP porque no forman parte del flujo de un pedido. Un ajuste manual tampoco puede dejar el stock en negativo.

5.7. Catálogo de hechos

Hecho	Quién lo publica	Quién reacciona
OrderPlaced	OrdersApi, al guardar el pedido.	InventoryApi reserva o rechaza.
StockReserved	InventoryApi, si había disponible.	OrdersApi pasa a awaiting-payment. PaymentsApi abre el pago pending.
StockRejected	InventoryApi, si no alcanzaba.	OrdersApi pasa a rejected.
PaymentConfirmed	PaymentsApi, simulación aprobada.	OrdersApi pasa a paid. InventoryApi descuenta stock y marca sold.
PaymentFailed	PaymentsApi, simulación rechazada.	OrdersApi pasa a payment-failed. InventoryApi libera la reserva.
OrderCancelled	OrdersApi, al cancelar.	InventoryApi libera la reserva. OrdersApi ignora hechos tardíos de ese pedido.

Tabla 3. Hechos del tópico shop-events y efecto en cada servicio.

5.8. Estados del pedido y de la reserva

Estado del pedido	Significado para la persona	Qué puede ocurrir después
pending-stock	Verificando inventario.	Pasar a awaiting-payment, a rejected o a cancelled.
awaiting-payment	Inventario reservado. Pendiente de pago.	Simular pago o cancelar.
rejected	Sin stock suficiente.	El flujo de esa compra termina.
paid	Pago confirmado y stock descontado.	El flujo de esa compra termina.
payment-failed	Pago rechazado y reserva liberada.	El flujo de esa compra termina.
cancelled	Cancelado antes del pago.	La reserva, si existía, se libera.

Tabla 4. Estados de ShopOrder y transiciones que el código permite.

La reserva, dentro de inventario, tiene su propio ciclo: `reserved` al apartar, `sold` si el pago se confirma y `released` si el pago falla o el pedido se cancela. El índice único de reserva por pedido y producto evita apartar dos veces la misma línea.

5.9. Lectura de una venta completa

Paso	Canal	Qué queda guardado
Ver relojes	HTTP, navegador e inventario	Nada nuevo.
Armar la cesta	Solo el navegador	La selección, hasta crear el pedido o recargar la página.
Crear el pedido	HTTP hacia pedidos; pedidos consulta inventario	Pedido pending-stock en orders.db.
Avisar la compra	Kafka, OrderPlaced	Mensaje en shop-events.
Apartar unidades	Kafka, StockReserved o StockRejected	Reserva, o el pedido rejected.
Actualizar el pedido y abrir el cobro	Los dos servicios leen Kafka	awaiting-payment y un pago pending.
Simular el pago	HTTP hacia pagos; pagos consulta el pedido; luego Kafka	Pago approved o rejected.
Cerrar la venta	Inventario y pedidos leen Kafka	Stock descontado o reserva liberada; pedido paid o payment-failed.

Tabla 5. Recorrido de una compra y el lugar donde queda cada resultado.

6. Datos que guarda cada servicio

Servicio	Archivo	Registros
InventoryApi	inventory.db	Product (stock y reserved), Reservation, StockMovement y ProcessedInventoryEvent.
OrdersApi	orders.db	ShopOrder, OrderItem (nombre y precio copiados) y ProcessedOrderEvent.
PaymentsApi	payments.db	Payment (un registro por pedido, con monto y estado) y ProcessedPaymentEvent.

Tabla 6. Dueño de cada dato. No hay tablas compartidas.

OrderItem guarda el nombre y el precio del momento de la compra. Si después el administrador cambia el precio del reloj, los pedidos ya creados conservan el valor con el que se cerraron. El movimiento de inventario guarda la fecha en UTC, el cambio (positivo entra, negativo sale) y el motivo.

7. Cómo se evita procesar dos veces el mismo hecho

Un mensaje puede leerse de nuevo si el offset no se confirmó. Cada trabajador hace lo mismo antes de aplicar una regla: busca el identificador del evento en su tabla. Si ya está, confirma el offset y sigue. Si no está, aplica el cambio, inserta el identificador y guarda. El índice único es la red de seguridad por si dos lecturas coincidieran.

Eso cubre la reentrega del mismo mensaje. No cubre dos pedidos HTTP distintos por la misma cesta: cada alta genera un pedido nuevo y un `OrderPlaced` nuevo. Tampoco sustituye la caja de salida descrita en las limitaciones. La idempotencia del productor, por su parte, cubre el reintento de envío hacia el broker, no la repetición de la petición del navegador.

8. Despliegue y forma de demostrarlo

Hace falta Docker con Compose v2 y el puerto 8088 libre. El primer arranque descarga la imagen de Kafka 3.9.1, la de Nginx 1.27 y los paquetes de .NET. El comando es `docker compose up --build -d`, ejecutado dentro de la carpeta TiendaRelojes. La tienda abre en http://localhost:8088. Los registros se ven con `docker compose logs -f`.

Una demostración de principio a fin sigue este orden:

- Abrir el catálogo y comprobar los cuatro relojes.
- Pulsar «Añadir a la cesta» y ver la línea, el total y el contador del encabezado.
- Completar nombre, teléfono, dirección y forma de pago, y crear el pedido.
- Esperar a que el seguimiento pase de «Verificando inventario» a «Pendiente de pago». Ese cambio es la prueba de que OrderPlaced y StockReserved ya circularon.
- Simular el pago aprobado. El pedido queda pagado y el stock de ese reloj baja una unidad. El movimiento aparece en administración.
- Repetir con «Simular pago rechazado». El pedido queda en payment-failed y el stock no baja.
- Pedir más unidades de las disponibles. Inventario publica StockRejected y el pedido queda rejected.

Esos recorridos están anotados en docs/PRUEBAS.md. En una ejecución observada, un reloj con 12 unidades pasó a 11 después del pago aprobado, y un pedido de 50 unidades sobre un reloj con 6 quedó rechazado.

9. Cierre

El sistema reutiliza la tienda y los tres microservicios del proyecto base, y los deja comunicados por un contrato común en Shared. Kafka lleva los hechos de la compra. HTTP se usa cuando hace falta una respuesta inmediata: el precio, el estado antes de simular el pago y todo lo que ve la persona. Cada servicio escribe solo en su base, reconoce los mensajes que ya aplicó y cambia el pedido o el stock únicamente en la transición que le corresponde.

Con eso se puede explicar una venta completa: la cesta no reserva, el pedido anuncia, el inventario aparta, el pago simulado decide y, solo entonces, la pieza sale o la reserva se devuelve. El prototipo es suficiente para el Tema 6 y deja explícito lo que faltaría para usarlo como una tienda real.